research · 2026-10-02

benchmarks are living things

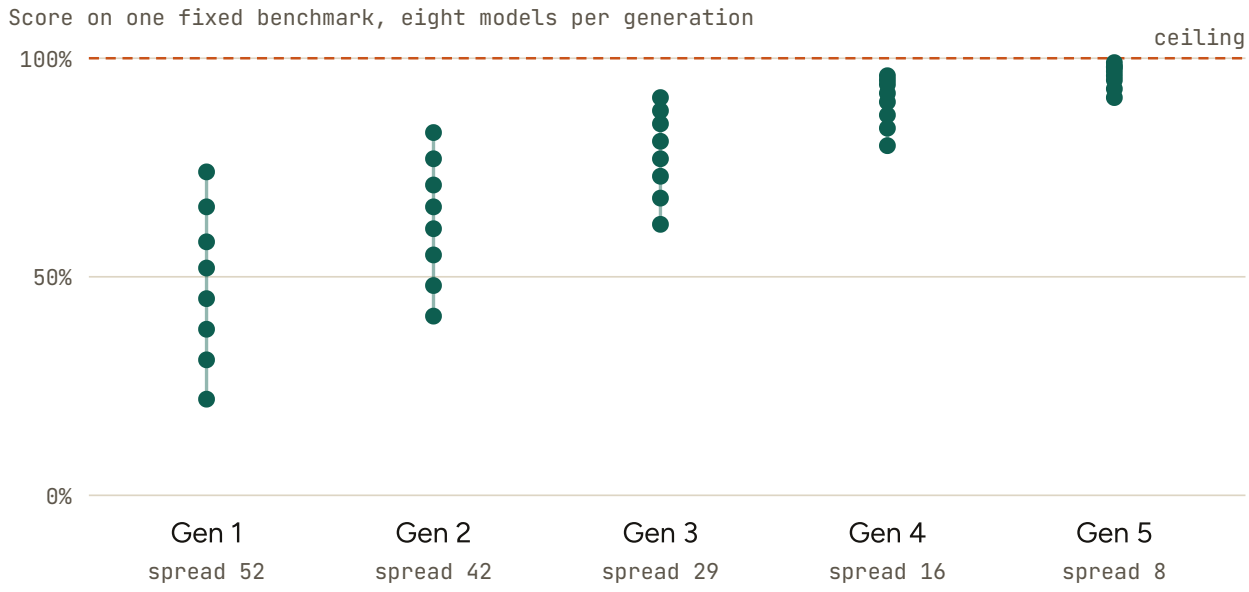
Adrian Tame, writing for plicara labs · 2026-10-02 ·

Evaluating AI systems keeps raising a problem I suspect many people have met: a benchmark where almost everything we put through it gets roughly the same high score. In document evaluation, for example, a collection of clean digital documents can get to the point where the models we're interested in all handle it quite well. That's good, obviously, but it leaves us with a problem if the reason we're running the benchmark is to understand the differences between those models. We can keep running it and reporting the results, but we're getting less out of it than we used to.

My first reaction to that particular problem is to find harder documents. Introduce scans, more complicated layouts like uneven tables, or something else that makes the models work a little harder, and see where they start to separate again. That makes sense if we're trying to understand their direct capabilities and where the limits are. But there's another interpretation, which fits what I've seen in practice: if these are the documents we actually need to process, maybe the models are already good enough. In that case, the more interesting differences might be what they cost or how well they fit into the system we're building. The same result can tell us to develop a harder benchmark or to stop paying for capability we don't need, depending on what we were trying to find out.

That's where this idea (not novel, of course) of benchmarks as living things comes from. Their usefulness depends on the systems they're evaluating and on the purpose of the evaluation, both of which can change while the benchmark itself stays exactly the same. I'd like to judge a benchmark by the information it contributes toward the purpose it was designed to serve, and then keep asking whether it is still doing that as the models improve. Choosing a model is one purpose, but so is investigating a failure, testing a capability or understanding whether a change to a system helped. The question is what we're learning from the result, and if there is enough useful information and variability between the candidates to eke out a statistically significant result.

FIGURE 1. SAME TASK, NEWER MODELS



Invented scores for eight imaginary models in each of five generations, on one unchanged benchmark. As the models improve, the spread between them shrinks toward the ceiling. Illustration, not measured data.

what are we still able to distinguish?

If I ask a collection of coding models to write `Hello, world!` in python, I probably won't learn very much about which one is better at programming. Suppose every model gets it right every time. We have a working evaluation, a clear scoring rule and a perfectly understandable task, but we've chosen a problem that doesn't distinguish the systems in front of us. That task might still be useful for checking that the execution environment works. As a comparison of coding capability, though, it doesn't get us very far.

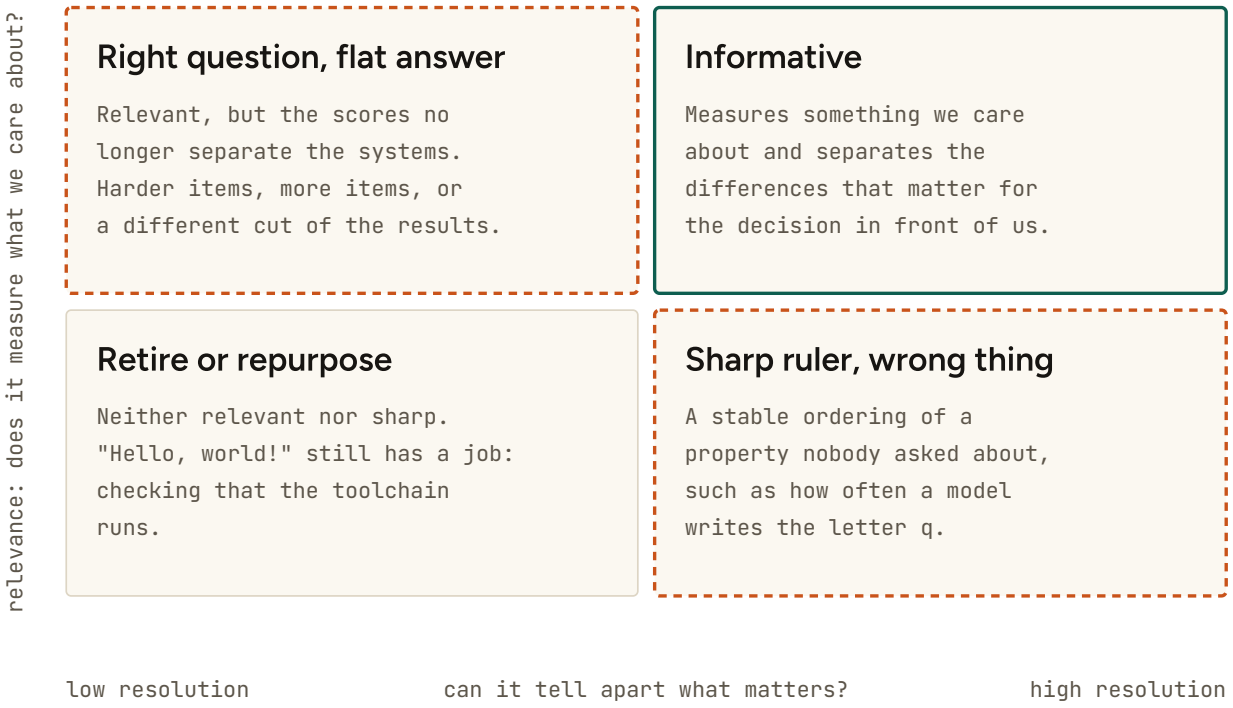
The same thing can happen with much more substantial benchmarks. A 2026 study by Akhtar and colleagues examined 60 large language model benchmarks and found that nearly half exhibited saturation, with saturation becoming more common as benchmarks aged.[1] Their analysis also associated expert curation with greater resilience to saturation, while keeping test data private did not offer the same protection. That finding separates two concerns that are easy to mix together: keeping the answers out of training data and keeping the questions informative as models improve.

There's also work that treats discrimination as something we can study directly. MetaEval estimates how well individual questions distinguish models and uses that information to select smaller subsets that retain much of the full benchmark's ranking and predictive value.[2] Zhou and colleagues approach benchmark quality through item response theory, which models the relationship between a test item's properties and the ability of the system answering it, and find problems with how well existing benchmarks measure differences between models.[3] These papers give us ways to check whether the questions we're asking are doing the correct work.

Discrimination alone doesn't settle it, though. I could rank models by how often they use the letter `q`, and even if that ordering were perfectly stable it wouldn't help me unless I had a reason to care about that particular property. So there are two things I keep coming back to: relevance, meaning that the benchmark measures something we care about, and resolution, meaning that it can distinguish the differences that matter for that purpose. If we're ranking models, those are differences between models. If we're checking a minimum standard, the distinction might be between acceptable and unacceptable performance.

Neither property belongs to the benchmark on its own. The same questions can separate smaller models while telling us little about more capable ones, or be useful for one kind of document and irrelevant to another. Calling a benchmark saturated without saying which systems we're evaluating leaves out part of the claim. Calling it useful without saying what we want to know kind of cancels out the usefulness. Benchmarks are not abstract objects that measure overall intelligence or capabilities, they can be correlated, but it is dangerous to conflate them.

FIGURE 2. RELEVANCE AND RESOLUTION

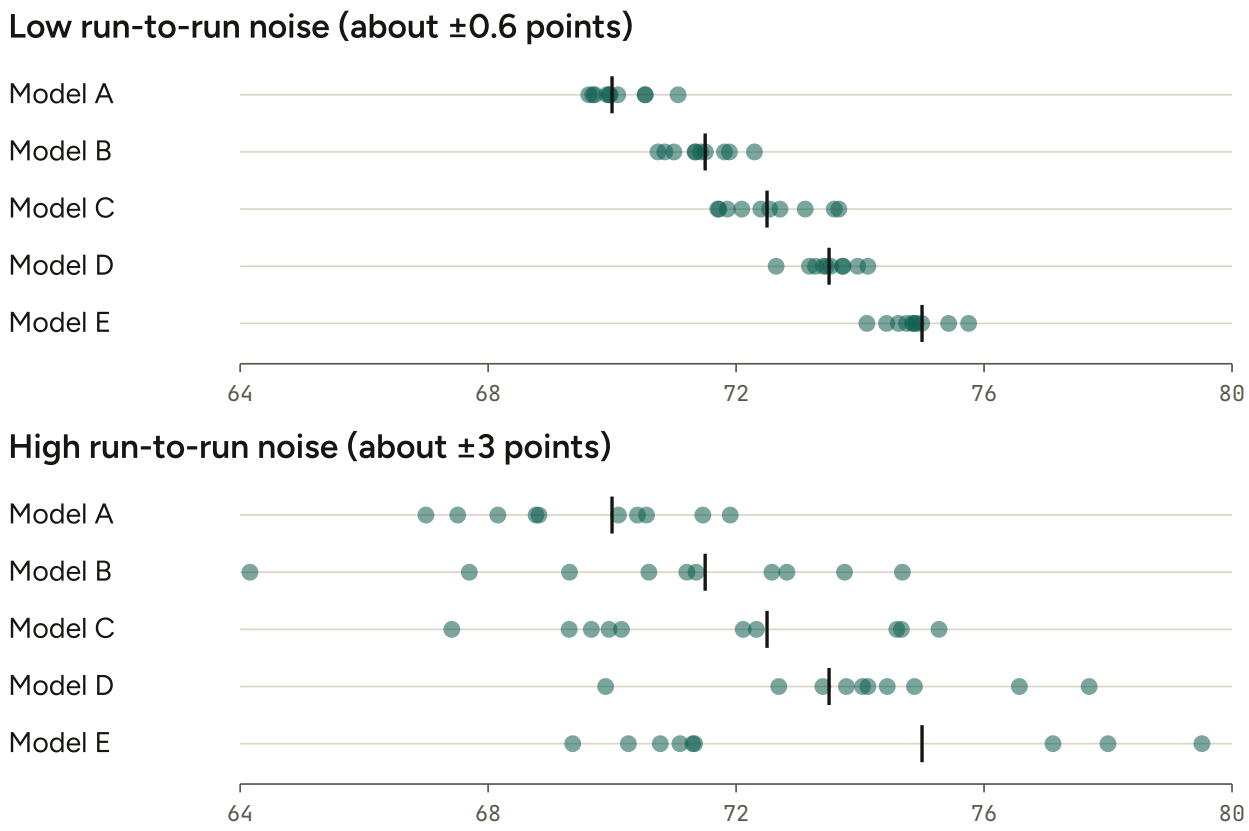


Two properties that neither belong to the benchmark alone: they depend on the systems being evaluated and the purpose of the evaluation. The examples are the ones used in the text.

a score comes with uncertainty

Something else that throws a wrench here is that the systems we're evaluating are stochastic. Suppose we run a benchmark ten times and the ordering changes so much that we can't tell whether one model is better or whether we're looking at variation between runs. There may still be a difference, and more measurements might help us estimate it, but the ranking from any single run isn't enough to establish one. Before deciding that a benchmark has stopped distinguishing models, we need to understand the uncertainty in the comparison and how large a difference would matter in the first place. I ran into this directly when a content filter blanked a large share of one model's answers and moved it seven places in our ranking; the write-up is the clearest example I have of why one run is not enough.

FIGURE 3. TEN RUNS OF FIVE MODELS



Each dot is one run; the dark tick is the true score.
The gaps between models are identical in both panels.

Simulated scores with a fixed seed: five imaginary models with the same true gaps, each run ten times on two benchmarks that differ only in run-to-run noise. With low noise one run is enough to rank them; with high noise the dots overlap and a single run cannot. Illustration, not measured data.

I find the language of information theory useful here because it makes us ask what changed after we saw the result. Are we more confident about which system can handle a task? Did we discover a failure we weren't aware of? Can we rule out an explanation we previously thought was plausible? Those are all things an evaluation can contribute, and they aren't interchangeable with getting another decimal place on a leaderboard. A result can be informative without producing a new winner.

I will say that I'm borrowing the intuition from information theory without claiming to have a single information-theoretic measure of benchmark quality. Repeated runs, uncertainty estimates and inspection of the errors can help us understand what a score tells us; they don't make relevance disappear as a judgment we still have to make. For this article, the claim is that a

benchmark should contribute information toward its purpose, and that we should pay attention when that contribution starts to diminish.

outgrowing a benchmark can be a good outcome

If a benchmark was created because models couldn't reliably do something, and later the models we're interested in can do it, I don't think we should automatically consider that a failure of the benchmark. It may have done exactly what we needed. It gave us a way to evaluate a capability while that capability was developing, and eventually the systems got good enough that the original questions stopped separating them. There is still work in establishing that this is what happened, because high scores can have other explanations, but a benchmark becoming less useful can be a consequence of progress.

That doesn't mean I'd ask someone building a new benchmark to specify its retirement date. We don't know what the models will look like by then, hell, we don't know what they will look like a month from now, or which parts of the task will turn out to be easy, or whether the purpose of the evaluation will change. The single most important place to start is with something we want to evaluate and a reason to evaluate it. The first version might need better questions before it distinguishes models well, which is fine as part of development. We have to start somewhere, and a real need gives the work a direction.

I would also be careful about calling the whole benchmark dead when scores converge or the like. A set of tasks that no longer separates leading models might still be useful for checking regressions or comparing systems under tighter resource constraints. Even `Hello, world!` has a job if we're checking whether the toolchain runs. What gets exhausted is a particular use of the benchmark for a particular population, and keeping that qualification in mind makes the lifecycle metaphor a lot less absolute.

the complexity has to come from somewhere

When the purpose is still to compare capabilities, increasing the complexity of the task is one way to restore resolution. This is the part of benchmark development that interests me most. A coding evaluation might move from writing an isolated function to making a change across a repository. An agent evaluation might start with a successful tool call and then ask the system to recover when a tool fails halfway through a longer task. In document understanding, clean pages might give way to scans, difficult tables or layouts where extracting the text is only part of understanding what it says.

All of those changes need a reason, though. We can make a benchmark harder by adding obscure trivia or awkward instructions, but unless those additions belong to the capability we care about, we may have made the evaluation worse. Going from clean digital documents to messy scans makes sense if the question includes messy scans. It doesn't automatically help if the workload consists entirely of clean digital documents. Difficulty gives us somewhere to look for differences; relevance determines whether those differences are worth looking for.

Dynamic benchmarks already put this into practice. Dynabench uses a process where people construct examples that a target model gets wrong but another person can answer, so model failures help determine what gets added.[4] LiveBench uses frequently updated questions and introduces harder tasks over time to keep distinguishing models as their capabilities improve.[5] These are different approaches, but both allow the evaluation to respond to what the systems can currently do. We don't have to discover every useful test case at the moment we release the first version.

There is a cost to that flexibility. If we change the questions, we also change what the score refers to, so a result from one version can't automatically be compared with a result from another. I'd want the versions preserved, the changes documented and some way of comparing systems on the same material. That might mean retaining common items or rerunning models on a new version. Otherwise, we could see a score fall after an update and have no clean way to tell how much came from harder questions and how much came from a change in the system.

sometimes good enough is what we needed to learn

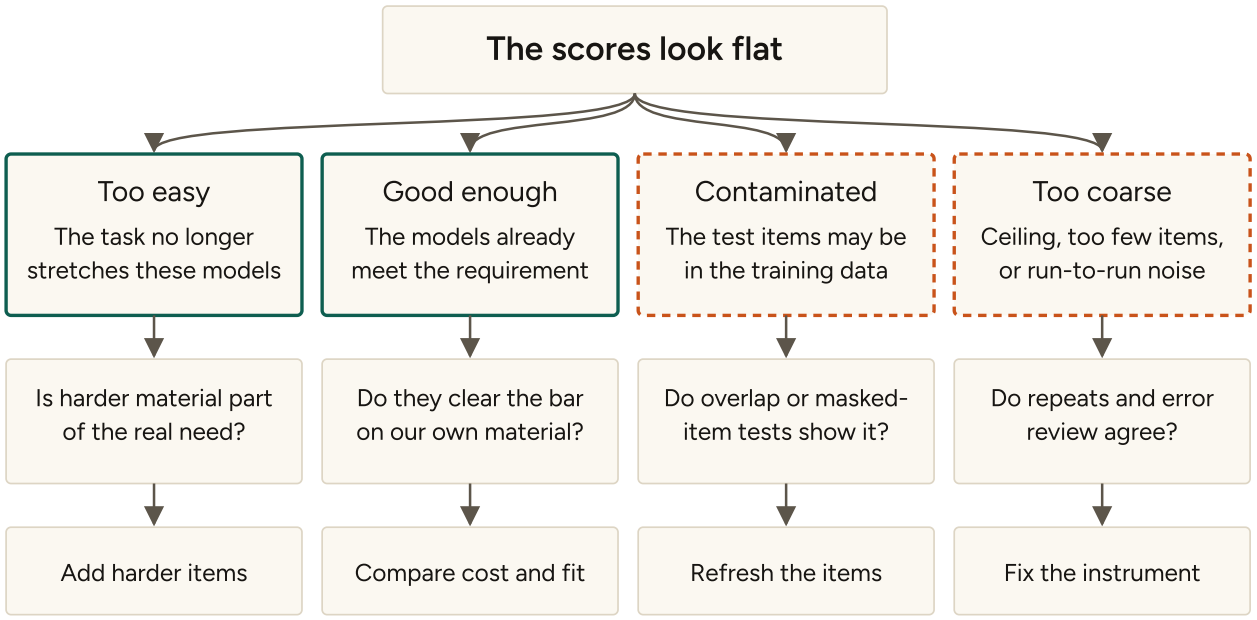
Returning to the document example, there's a point where adding harder material would answer a different question from the one we started with. If the documents we need to process are straightforward and several models handle them well enough, it makes sense to compare those models on cost, latency or integration. I care about this with coding models too: paying more for a model only makes sense if the extra capability is doing something useful for the work I'm asking it to do. For coding models though, I've found that I always appreciate the extra capability, even if it is not the case with document extraction or OCR workloads.

For a hypothetical example, suppose two models meet the quality requirement for a task, with enough evaluation to support that conclusion, and one costs a tenth as much to run. We still have to account for the rest of the system, including retries and the cost of failures, but the capability ranking may no longer be the most useful comparison. The benchmark has helped establish the condition under which we can consider the cheaper model. That is worth knowing even if the

scores don't separate the models, if anything, it matters because the benchmark can't separate the models.

The qualification is that similar scores don't establish equivalent capability. They might reflect a ceiling in the scoring rule, too few examples, or failures that cancel each other out in the average. Before treating several models as good enough, we need evidence that they clear the requirement on the material we care about. Once we have that, though, a compressed leaderboard can be a reason to change what we optimize for.

FIGURE 4. FOUR THINGS A FLAT RESULT CAN MEAN



Solid outline: the benchmark may still be doing its job. Dashed: the score itself needs repair.

The same flat leaderboard can call for a harder benchmark, a different comparison, fresh items or a better instrument. The question asked of the result decides which. This is a decision aid drawn from the essay, not a flowchart every benchmark must follow.

The move from capability to cost is the one I'd expect to see most often. Other changes in emphasis, toward maintainability or operational risk, for example, seem plausible but will depend on the system. I wouldn't try to turn those into stages that every benchmark goes through. What interests me is that the result can change the next question we need to ask.

what the score actually measures

A benchmark doesn't have to saturate to become less trustworthy. If its test examples appear in training data, a model's performance may partly reflect exposure to those examples, which makes the result harder to interpret as evidence of generalization. Sainz and colleagues describe this problem directly in their work on contamination in language model evaluation.[6] The benchmark can still produce a clean ranking while leaving us less certain about what that ranking measures.

There are several approaches to investigating it. Deng and colleagues look for overlap with training corpora and test whether models can recover deliberately hidden parts of benchmark examples.[7] Zhu and colleagues propose detecting and rewriting leaked examples at evaluation time, while Choi and colleagues propose a method for measuring leakage using changes in representations after fine-tuning.[8][9] Those methods address different parts of the problem; their existence doesn't mean we can take an arbitrary benchmark score and cleanly subtract the contribution of memorization.

For the argument here, contamination matters because the relevance of the task and the interpretation of its score can come apart. We may still care about exactly the same capability, but no longer have confidence that the evaluation measures it in the way we intended. That is different from all the models becoming good at the task, even if both problems eventually lead us to replace some questions. Looking only at the spread of scores won't tell us which problem we have.

There is another judgment involved even when the data is clean: what counts as a good answer. This comes up immediately with explanations, code and agent behavior, where several outputs can be correct without being equally useful. How much detail belongs in an explanation, which mistakes deserve the largest penalties, and whether a working solution is understandable enough to maintain are choices we make while designing the evaluation. Even a benchmark with an objective scoring rule contains decisions about which tasks to include and how much each one counts.

The Holistic Evaluation of Language Models project, HELM, makes a case for evaluating across multiple scenarios and metrics, while being explicit about what the evaluation leaves out.[10] I like that framing because it makes those choices visible. A benchmark represents some view of what matters, and we should be able to examine that view along with the scores. As models are developed against an evaluation, it is also worth checking whether improvements on its metric still correspond to improvements in the thing we wanted. We can optimize a proxy quite effectively without resolving that question.

more benchmarks can leave us with the same uncertainty

Most of this discussion has been about a single benchmark, but the same question applies to a collection of them. Suppose several benchmarks produce almost identical rankings over the same models. That may be useful confirmation, or it may mean we're paying to measure much of the same variation repeatedly. The number of columns in the results table doesn't tell us which of those is happening, and treating correlated measurements as independent evidence can make a conclusion look better supported than it is.

Work on redundancy in multimodal benchmarks examines this at the level of capability dimensions, individual questions and entire benchmarks.[11] BenchBench studies how methodological choices affect conclusions about agreement between benchmarks.[12] Both are relevant to the question I'd want to ask when adding another evaluation: what does this help us learn that we couldn't already learn from the ones we're running?

Correlation alone isn't enough to decide that a benchmark is redundant. Two evaluations can agree on the ordering while exposing different failure cases, and another measurement can improve precision even if it doesn't change the winner. But if we're adding cost and more results to interpret without meaningfully changing what we know, that's worth addressing. The information a benchmark contributes alongside other benchmarks deserves its own article, particularly the difference between useful corroboration and repeating a measurement under several names.

what maintaining a benchmark means

For me, this changes what it means to maintain an evaluation. Keeping the code running is part of it, but so is checking whether the questions still distinguish what we need them to distinguish, whether the errors still matter and whether we can still interpret the score. When results start to compress, I'd want to inspect where the models fail and how much the comparison varies before deciding to make the benchmark harder. When exposure to test data becomes a concern, the question is whether we can still support the conclusions we're drawing from it.

Those responses won't be the same for every benchmark. Some will need more complex tasks, some will need fresh examples, and some should keep their questions because checking an established capability is still useful. In other cases, the evaluation may have given us enough evidence to move on to cost or another property of the system. The purpose tells us which response makes sense, and the results tell us when we need to reconsider it.

That's what I mean by benchmarks being living things. We start with something we want to understand, build a way to evaluate it, and keep learning about both the models and the evaluation as we use it. I don't expect the first set of questions to remain the right set indefinitely. I do expect us to notice when they stop helping, and to be willing to change them when that happens.

references

1. Mubashara Akhtar et al. When AI Benchmarks Plateau: A Systematic Study of Benchmark Saturation. ICML, 2026. ↩

2. Zhuo Wang, Wen Wu, Guoqing Wang, Guangze Ye and Zhenxiao Cheng. MetaEval: Measuring the Discrimination of Benchmarks for Efficient LLM Evaluation. AAAI, 2026. ↩

3. Hongli Zhou et al. Lost in Benchmarks? Rethinking Large Language Model Benchmarking with Item Response Theory. AAAI, 2026. ↩

4. Douwe Kiela et al. Dynabench: Rethinking Benchmarking in NLP. NAACL, 2021. ↩

5. Colin White et al. LiveBench: A Challenging, Contamination-Limited LLM Benchmark. ICLR, 2025. ↩

6. Oscar Sainz, Jon Campos, Iker García-Ferrero, Julen Etxaniz, Oier Lopez de Lacalle and Eneko Agirre. NLP Evaluation in Trouble: On the Need to Measure LLM Data Contamination for Each Benchmark. Findings of EMNLP, 2023. ↩

7. Chunyuan Deng, Yilun Zhao, Xiangru Tang, Mark Gerstein and Arman Cohan. Investigating Data Contamination in Modern Benchmarks for Large Language Models. NAACL, 2024. ↩

8. Qin Zhu et al. Inference-Time Decontamination: Reusing Leaked Benchmarks for Large Language Model Evaluation. Findings of EMNLP, 2024. ↩

9. Hyeong Kyu Choi, Maxim Khanov, Hongxin Wei and Yixuan Li. How Contaminated Is Your Benchmark? Measuring Dataset Leakage in Large Language Models with Kernel Divergence. ICML, 2025. ↩

10. Stanford CRFM. Holistic Evaluation of Language Models. Project introduction, 2022. ↩

11. Zicheng Zhang et al. Redundancy Principles for MLLMs Benchmarks. ACL, 2025. ↩

12. Yotam Perlitz et al. Do These LLM Benchmarks Agree? Fixing Benchmark Evaluation with BenchBench. Preprint, 2024. ↩